

AI534 — Written Homework Assignment 2 (45 pts) — Manogna Challoju

This assignment covers Kernel methods and Support vector machines.

1. (Cubic Kernels.) (8 pts) In class, we showed that the quadratic kernel $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + 1)^2$ was equivalent to mapping each $\mathbf{x} = (x_1, x_2) \in R^2$ into a higher dimensional space where

$$\Phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2, \sqrt{2}x_1, \sqrt{2}x_2, 1).$$

Now consider the cubic kernel $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + 1)^3$. What is the corresponding Φ function?

Given that $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + 1)^2$ where $\mathbf{x} = (x_1, x_2) \in R^2$

$$\Phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2, \sqrt{2}x_1, \sqrt{2}x_2, 1).$$

To find Φ for the cubic kernel $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + 1)^3$ The first step is to expand the expression $(\mathbf{x}_i \cdot \mathbf{x}_j + 1)^3$

Let $\mathbf{x} = (x_1, x_2) \in R^2$.

$$(\mathbf{x}_i \cdot \mathbf{x}_j + 1)^3 = (\mathbf{x}_i \cdot \mathbf{x}_j)^3 + 3(\mathbf{x}_i \cdot \mathbf{x}_j)^2 + 3(\mathbf{x}_i \cdot \mathbf{x}_j) + 1.$$

To find $\Phi(\mathbf{x})$, we need to express each term that would appear if we multiplied out $(x_1x'_1 + x_2x'_2 + 1)^3$

Which can be expanded using the binomial theorem:

$$(x_1x'_1 + x_2x'_2 + 1)^3 = (x_1x'_1)^3 + 3(x_1x'_1)^2(x_2x'_2) + 3(x_1x'_1)(x_2x'_2)^2 + (x_2x'_2)^3 + 3(x_1x'_1)^2 + 3(x_2x'_2)^2 + 3(x_1x'_1 + x_2x'_2) + 1.$$

Terms with third powers : x_1^3, x_2^3

Terms with squared terms and one mixed term: $x_1^2x_2, x_2^2x_1$

Terms with linear terms and two mixed terms: $x_1x_2^2, x_2x_1^2$

Terms with linear terms and constant: $\sqrt{3}x_1, \sqrt{3}x_2$

Constant term: 1

$\Phi(\mathbf{x})$: Φ function for the cubic kernel is:

$$\Phi(\mathbf{x}) = (x_1^3, x_2^3, x_1^2x_2, x_2^2x_1, x_1x_2^2, x_2x_1^2, \sqrt{3}x_1, \sqrt{3}x_2, 1).$$

2. (Kernel or not). (5 pts) Suppose that K_1 and K_2 are kernels with feature maps ϕ_1 and ϕ_2 respectively. Is function $K(\mathbf{x}, \mathbf{z}) = c_1K_1(\mathbf{x}, \mathbf{z}) + c_2K_2(\mathbf{x}, \mathbf{z})$ for $c_1, c_2 > 0$ a kernel function? If your answer is yes, write down the corresponding ϕ in terms of ϕ_1 and ϕ_2 . If not, provide a proof or explain why.

Yes $K(\mathbf{x}, \mathbf{z}) = c_1K_1(\mathbf{x}, \mathbf{z}) + c_2K_2(\mathbf{x}, \mathbf{z})$ is a valid kernel function.

As K_1 and K_2 are kernel functions, they represent valid inner products in their feature spaces.

$$K_1(\mathbf{x}, \mathbf{z}) = \langle \phi_1(\mathbf{x}), \phi_1(\mathbf{z}) \rangle$$

$$K_2(\mathbf{x}, \mathbf{z}) = \langle \phi_2(\mathbf{x}), \phi_2(\mathbf{z}) \rangle.$$

Also c_1 and c_2 are positive constants, the linear combination $c_1K_1(\mathbf{x}, \mathbf{z}) + c_2K_2(\mathbf{x}, \mathbf{z})$ is also a valid inner product in the combined feature space.

The combined kernel function $K(\mathbf{x}, \mathbf{z})$ can be show by a feature map ϕ defined as the concatenation of ϕ_1 and ϕ_2 , scaled by $\sqrt{c_1}$ and $\sqrt{c_2}$

$$\phi(\mathbf{x}) = (\sqrt{c_1}\phi_1(\mathbf{x}), \sqrt{c_2}\phi_2(\mathbf{x})).$$

The inner product in the combined feature space is:

$$\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle = \langle \sqrt{c_1} \phi_1(\mathbf{x}), \sqrt{c_1} \phi_1(\mathbf{z}) \rangle + \langle \sqrt{c_2} \phi_2(\mathbf{x}), \sqrt{c_2} \phi_2(\mathbf{z}) \rangle.$$

$$\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle = c_1 K_1(\mathbf{x}, \mathbf{z}) + c_2 K_2(\mathbf{x}, \mathbf{z}).$$

$K(\mathbf{x}, \mathbf{z}) = c_1 K_1(\mathbf{x}, \mathbf{z}) + c_2 K_2(\mathbf{x}, \mathbf{z})$ is a valid kernel with the feature map $\phi(\mathbf{x}) = (\sqrt{c_1} \phi_1(\mathbf{x}), \sqrt{c_2} \phi_2(\mathbf{x}))$.

3. Kernelizing Logistic Regression (10 pts) For this problem you will follow the example of kernelizing perceptron, to kernelize the logistic regression shown below.

Algorithm 1: Stochastic gradient descent for logistic regression

Input: $\{(\mathbf{x}_i, y_i)_{i=1}^N\}$ (training data), γ (learning rate)

Output: learned weight vector $\mathbf{w}$

```

1 Initialize  $\mathbf{w} = \mathbf{0}$ ;
2 while not converged do
3   for  $i = 1, \dots, N$  do
4      $\mathbf{w} \leftarrow \mathbf{w} + \gamma(y_i - \sigma(\mathbf{w}^T \mathbf{x}_i)) \mathbf{x}_i$ 
5   end
6 end
```

Specifically, please:

- (a) (4 pts) Argue that the solution $\mathbf{w}^*$ for logistic regression can be expressed as the weighted sum of training examples (similar to slide 8 of the kernel methods lecture)

Given that $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + 1)^3$, and to kernelize the logistic regression algorithm using the ideas presented for the kernelized perceptron -

The weight vector $\mathbf{w}$ in logistic regression is updated iteratively through:

$$\mathbf{w} \leftarrow \mathbf{w} + \gamma(y_i - \sigma(\mathbf{w}^T \mathbf{x}_i)) \mathbf{x}_i,$$

where $\sigma(\mathbf{w}^T \mathbf{x}_i) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}_i}}$ is the sigmoid function.

Each update to $\mathbf{w}$ involves adding a term proportional to the training example $\mathbf{x}_i$, showing that $\mathbf{w}$ is gradually built as a linear combination of training data points so, after T iterations, it can be expressed $\mathbf{w}$ as:

$$\mathbf{w} = \sum_{i=1}^N \alpha_i \mathbf{x}_i,$$

where each α_i gathers contributions from $\mathbf{x}_i$ over all updates and depends on γ and the terms $(y_i - \sigma(\mathbf{w}^T \mathbf{x}_i))$ across iterations.

This results in the final weight vector $\mathbf{w}^*$, which can be represented as:

$$\mathbf{w}^* = \sum_{i=1}^N \alpha_i \mathbf{x}_i,$$

- (b) (6 pts) Modify the following stochastic gradient descent algorithm logistic regression algorithm to kernelize it. (Hint: similar to the bottom algorithm on slide 14 of the kernel method lecture, but instead of counter, you will learn a continuous weights for α 's)

The kernelized version of the stochastic gradient descent algorithm for logistic regression is

Algorithm 2: Kernelized Stochastic Gradient Descent for Logistic Regression

Input: $\{(\mathbf{x}_i, y_i)_{i=1}^N\}$ (training data), γ (learning rate)

Output: learned weight vector in terms of α

```
1 Initialize  $\alpha_i = 0$  for all  $i = 1, \dots, N$ ;  
2 while not converged do  
3   for  $i = 1, \dots, N$  do  
4      $\hat{y}_i \leftarrow \sigma\left(\sum_{j=1}^N \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i)\right)$ ;  
5      $\alpha_i \leftarrow \alpha_i + \gamma(y_i - \hat{y}_i)$ ;  
6   end  
7 end
```

Breakdown of the Kernelized Algorithm

1. First we initialize each $\alpha_i = 0$ for all training examples $i = 1, \dots, N$.
2. For each training example $\mathbf{x}_i$, we calculate the predicted output $\hat{y}_i$ using the kernel function $K(\mathbf{x}_j, \mathbf{x}_i)$, which is the similarity between $\mathbf{x}_j$ and $\mathbf{x}_i$:

$$\hat{y}_i \leftarrow \sigma\left(\sum_{j=1}^N \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i)\right),$$

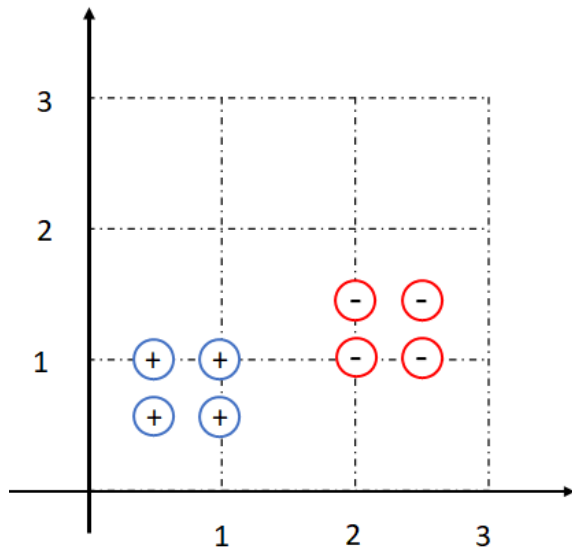
where σ is the sigmoid function. This step replaces the direct calculation of $\mathbf{w}^T \mathbf{x}_i$ with a kernelized form.

3. Then we update the weight α_i for each training example i based on the difference between the true label y_i and the predicted output $\hat{y}_i$:

$$\alpha_i \leftarrow \alpha_i + \gamma(y_i - \hat{y}_i).$$

By using this kernelized approach, we avoid explicitly computing $\mathbf{w}$ and instead work with kernel functions. This allows the algorithm to operate in a higher-dimensional feature space implicitly, through the kernel $K(\mathbf{x}_j, \mathbf{x}_i)$.

4. (Hard margin SVM) (8 pts) Apply linear SVM without soft margin to the following problem.



- a. (3pts) Please mark out the support vectors, the decision boundary ($w_1x_1 + w_2x_2 + b = 0$) and $w_1x_1 + w_2x_2 + b = 1$ and $w_1x_1 + w_2x_2 + b = -1$. You don't need to solve the optimization problem for this, you should be able to eyeball the solution and find the linear separator with the largest margin.

The support vectors are dark circled and the decision boundaries are marked in Figure 1

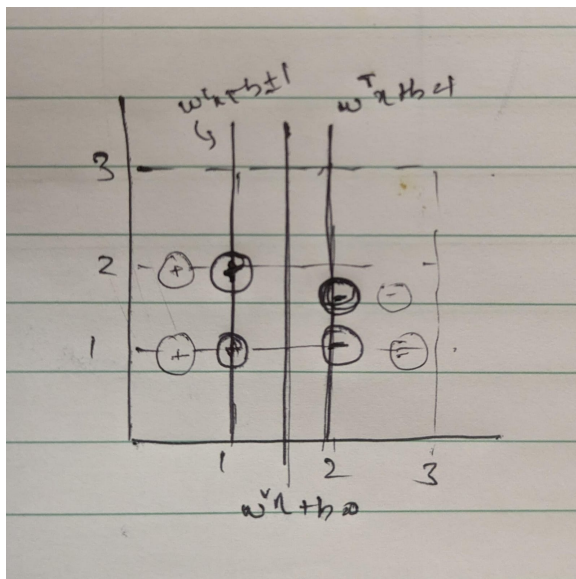


Figure 1: Marked SVM Decision Boundary

- The support vectors are the closest points of each class to the decision boundary.
 - * The blue points at (1,1) and (1,2) are the closest blue points to the boundary.
 - * The red points at (2,1) and (2,2) are the closest red points to the boundary.

- The decision boundary for an SVM is a line that maximizes the margin between the two classes. By visual inspection, we can place the boundary vertically between the blue and red clusters, approximately at $x = 1.5$.
- The margin lines represent boundaries on either side of the decision boundary, where the support vectors lie.
 - * The line $w_1x_1 + w_2x_2 + b = 1$ will be approximately at $x = 2$.
 - * The line $w_1x_1 + w_2x_2 + b = -1$ will be approximately at $x = 1$.
 - * Decision Boundary: A vertical line at $x = 1.5$.
 - * Margin Lines: Parallel lines at $x = 1$ and $x = 2$.
 - * Support Vectors: Points at $(1,1)$, $(1,2)$, $(2,1)$, and $(2,2)$.

Decision Boundary (solid line): $w_1x_1 + w_2x_2 + b = 0$, at $x = 1.5$.

Margin Lines (dashed lines):

- $w_1x_1 + w_2x_2 + b = 1$ at $x = 2$.
- $w_1x_1 + w_2x_2 + b = -1$ at $x = 1$.

Support Vectors: Points at $(1,1)$, $(1,2)$, $(2,1)$, and $(2,2)$.

- b. (5 pts) Please solve for w_1, w_2 and b based on the support vectors you identified in (a). Hint: the support vectors would have functional margin = 1.

The support vectors satisfy the following equations:

For the positive support vectors:

$$w_1 \cdot 1 + w_2 \cdot 1 + b = 1 \Rightarrow w_1 + w_2 + b = 1$$

$$w_1 \cdot 1 + w_2 \cdot 2 + b = 1 \Rightarrow w_1 + 2w_2 + b = 1$$

For the negative support vectors:

$$w_1 \cdot 2 + w_2 \cdot 1 + b = -1 \Rightarrow 2w_1 + w_2 + b = -1$$

$$w_1 \cdot 2 + w_2 \cdot 2 + b = -1 \Rightarrow 2w_1 + 2w_2 + b = -1$$

This system of equations is solved to obtain:

$$w_1 = -2, \quad w_2 = 0, \quad b = 3$$

Therefore the decision boundary equation is:

$$-2x_1 + 3 = 0$$

or equivalently,

$$x_1 = 1.5$$

which matches the visual assessment of the decision boundary at $x = 1.5$.

5. L_2 SVM (14 pts)

Given a set of training examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $y_i \in \{1, -1\}$ for all i . The following is the primal formulation of L_2 SVM, a variant of the standard SVM obtained by squaring the slacks.

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \mathbf{w}^T \mathbf{w} + c \sum_{i=1}^N \xi_i^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i \in \{1, \dots, N\} \\ & \xi_i \geq 0, \quad i \in \{1, \dots, N\} \end{aligned}$$

- a. (3pts) Show that removing the second constraint $\xi_i \geq 0$ will not change the solution to the problem. In other words, let $(\mathbf{w}^*, b^*, \xi^*)$ be the optimal solution to the problem without this set of constraints, show that $\xi_i^* \geq 0$ must be true, $\forall i \in \{1, \dots, N\}$. (Hint: use proof by contradiction by assuming that there exists some $\xi_i^* < 0$.)

To show that removing the second constraint $\xi_i \geq 0$ doesn't change the solution to the L2-SVM problem, we'll use proof by contradiction.

$$\min_{w, b, \xi} \frac{1}{2} w^T w + c \sum_{i=1}^N \xi_i^2$$

subject to: 1. $y_i(w^T x_i + b) \geq 1 - \xi_i$ for $i = 1, \dots, N$ 2. $\xi_i \geq 0$ for $i = 1, \dots, N$

To show that removing the second constraint, $\xi_i \geq 0$, doesn't affect the solution, meaning that in the optimal solution, ξ_i will always be non-negative even if we don't enforce it.

Proof by Contradiction

Suppose that in the optimal solution without the $\xi_i \geq 0$ constraint, there exists at least one slack variable ξ_i^* such that $\xi_i^* < 0$. Let's call this particular value ξ_k^* for some $k \in \{1, \dots, N\}$.

Impact of a Negative ξ_i on the Objective: Recall that the objective function is:

$$\frac{1}{2} w^T w + c \sum_{i=1}^N (\xi_i)^2$$

If $\xi_k^* < 0$, then $(\xi_k^*)^2$ is positive, because squaring any real number makes it non-negative. This squared term contributes positively to the objective function.

Creating a Lower Objective by Setting $\xi_k = 0$: If we set $\xi_k = 0$ (or any non-negative value), $(\xi_k)^2$ would be zero or lower than $(\xi_k^*)^2$, which is positive. This would result in a lower value for the objective function.

By setting $\xi_k = 0$, we would achieve a lower objective value than with $\xi_k^* < 0$. This contradicts our assumption that (w^*, b^*, ξ^*) is the optimal solution without the $\xi_i \geq 0$ constraint, because we found a better solution with $\xi_k \geq 0$.

Therefore, for the objective function to be minimized, each ξ_i must satisfy $\xi_i \geq 0$ naturally, even without enforcing this constraint explicitly.

Removing the constraint $\xi_i \geq 0$ does not affect the solution because any negative ξ_i would lead to a higher objective value, contradicting optimality. Thus, $\xi_i \geq 0$ will hold in the optimal solution regardless.

- b. (3 pts) After removing the second set of constraints, we have a simpler problem with only one set of constraints. Now provide the lagrangian of this new problem.

To create the Lagrangian for the simplified problem, start with the updated objective and constraint:

The optimization problem now is:

$$\min_{w, b, \xi} \frac{1}{2} w^T w + c \sum_{i=1}^N \xi_i^2$$

subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \text{for } i = 1, \dots, N$$

For each constraint $y_i(w^T x_i + b) \geq 1 - \xi_i$, introduce a Lagrange multiplier $\alpha_i \geq 0$.

The Lagrangian $\mathcal{L}(w, b, \xi, \alpha)$ becomes:

$$\mathcal{L}(w, b, \xi, \alpha) = \frac{1}{2} w^T w + c \sum_{i=1}^N \xi_i^2 - \sum_{i=1}^N \alpha_i (y_i(w^T x_i + b) - 1 + \xi_i)$$

This expression represents the Lagrangian for the new problem, where $\alpha_i \geq 0$ enforces the constraint for each training example.

- c. (8pts) Derive the dual of this problem. How is it different from the standard SVM with hinge loss? Which formulation is more sensitive to outliers?

$$\min_{w, b, \xi} \frac{1}{2} w^T w + c \sum_{i=1}^N \xi_i^2$$

$$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \text{for } i = 1, \dots, N$$

Using the Lagrange multipliers $\alpha_i \geq 0$ for each constraint $y_i(w^T x_i + b) \geq 1 - \xi_i$, the Lagrangian $\mathcal{L}(w, b, \xi, \alpha)$ is:

$$\mathcal{L}(w, b, \xi, \alpha) = \frac{1}{2} w^T w + c \sum_{i=1}^N \xi_i^2 - \sum_{i=1}^N \alpha_i (y_i(w^T x_i + b) - 1 + \xi_i)$$

Step 3: Find the Dual by Minimizing the Lagrangian with Respect to Primal Variables The dual function is obtained by minimizing the Lagrangian with respect to w , b , and ξ .

Differentiate with Respect to w :

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{i=1}^N \alpha_i y_i x_i = 0$$

This gives:

$$w = \sum_{i=1}^N \alpha_i y_i x_i$$

Differentiate with Respect to b :

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^N \alpha_i y_i = 0$$

This

$$\sum_{i=1}^N \alpha_i y_i = 0$$

Differentiate with Respect to ξ_i :

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 2c\xi_i - \alpha_i = 0$$

$$\xi_i = \frac{\alpha_i}{2c}$$

Step 4: Substitute Back into the Lagrangian Substitute $w = \sum_{i=1}^N \alpha_i y_i x_i$ and $\xi_i = \frac{\alpha_i}{2c}$ into $\mathcal{L}$ to eliminate w and ξ_i :

$$\mathcal{L} = \frac{1}{2} \left(\sum_{i=1}^N \alpha_i y_i x_i \right)^T \left(\sum_{j=1}^N \alpha_j y_j x_j \right) + c \sum_{i=1}^N \left(\frac{\alpha_i}{2c} \right)^2 - \sum_{i=1}^N \alpha_i \left(y_i \left(\sum_{j=1}^N \alpha_j y_j x_j \right) + b - 1 + \frac{\alpha_i}{2c} \right)$$

Expanding and simplifying the terms yields the dual formulation:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j - \frac{1}{4c} \sum_{i=1}^N \alpha_i^2$$

subject to:

$$\sum_{i=1}^N \alpha_i y_i = 0, \quad \alpha_i \geq 0 \quad \forall i$$

The dual formulation for L2-SVM differs from the standard SVM (which uses hinge loss) in the objective function. the $\frac{1}{4c} \sum_{i=1}^N \alpha_i^2$ term. the hinge loss introduces linear slack terms ξ_i in the primal form, leading to a simpler quadratic term without α_i^2 in the dual. The L2-SVM formulation (with squared slack terms) is more sensitive to outliers than the standard SVM. Since errors are squared, large deviations result in excessively large penalties, causing the model to be more influenced by outliers compared to the hinge loss used in standard SVM, which penalizes errors linearly and is more robust to such cases.